Smart Commerce - Viva Questions and Answers

Prepared from discussed supervisor questions, chatbot questions, and project defense answers

Use this document for viva, supervisor questions, and project defense preparation.

1. Question: What do you mean by Inventory Intelligence? How does it work?

Answer: Inventory Intelligence means smart stock management using product stock data, reorder points, sales information, and dashboard alerts. It helps the admin or vendor understand which products are available, low in stock, out of stock, or need restocking. The system compares current stock with the reorder point. If stock is zero, it shows out of stock. If stock is less than or equal to the reorder point, it shows low stock. Otherwise, it shows in stock.

2. Question: How did you implement Inventory Intelligence?

Answer: I implemented it by storing stock and reorderPoint for every product, creating an inventory overview API, showing stock status in admin and vendor dashboards, supporting stock adjustment, and saving inventory movement logs. When stock is adjusted, the system records previous stock, new stock, quantity change, reason, and user information.

3. Question: How is the low-stock alert given?

Answer: The low-stock alert is generated by comparing current stock with the reorder point or alert threshold. If stock is zero, the product is marked as out of stock or critical. If stock is less than or equal to the reorder point, it is marked as low stock. In the admin alert section, products below 7 units are also highlighted.

4. Question: Which technologies are used to identify low stock and show notification?

Answer: For identification, I used Node.js, Express.js, TypeScript, MongoDB, and Mongoose. MongoDB stores stock and reorder point values, while the backend checks them using rule-based conditions. For the notification UI, I used Next.js, React, Tailwind CSS, dashboard alert lists, badge components, and toast messages.

5. Question: How did you implement the chatbot? How does it work?

Answer: I implemented the chatbot as an AI-style customer support chatbot using Next.js, React, TypeScript, and an API route. The chatbot takes the user's question, sends it to /api/chatbot, normalizes the text, fixes common spelling mistakes, retrieves relevant project or product information, and returns the best answer in the chat widget.

6. Question: How can you describe the AI-based customer support chatbot?

Answer: The chatbot provides instant support for customers and users. It can answer questions about product price, stock, category, sales, checkout, order tracking, PDF receipt, dashboard features, inventory, low-stock alert, demand forecasting, demo accounts, and project setup. This reduces repeated manual support work and improves user experience.

7. Question: Can you say the chatbot uses RAG and Gemini API?

Answer: I should not claim Gemini API unless it is configured and actively used in the final running system. The current working version is a local RAG-style chatbot. It retrieves context from existing project and product data and answers from that context. Gemini API can be added in the future as the generation model.

8. Question: Is ChromaDB necessary for a RAG chatbot?

Answer: No, ChromaDB is not necessary. ChromaDB is only one option for vector storage. A RAG system mainly needs a knowledge source, retrieval logic, and answer generation. For this project, product data and project knowledge are stored locally in the code/data source, and retrieval is done using keyword and scoring logic.

9. Question: What is used to implement demand forecasting?

Answer: Demand forecasting uses Node.js, Express.js, TypeScript, MongoDB, Mongoose, a Python forecasting script, Next.js, React, and Recharts. The backend reads active products and previous non-cancelled orders, groups sales by product into weekly history, sends the data to Python, and receives predicted units, trend label, confidence score, keywords, and forecast values.

10. Question: Is the demand forecasting deep learning?

Answer: No, the current demand forecasting is not deep learning. It is an explainable forecasting method based on previous sales history, weighted average, trend analysis, keyword boost, and confidence scoring. This method is suitable for the project level and easy to explain.

11. Question: How did you build the chatbot? Which technologies are used?

Answer: The chatbot was built with Next.js, React, TypeScript, Tailwind CSS, and a Next.js API route. It uses a local project knowledge base, product dataset, text normalization, spelling correction, keyword matching, scoring, and local RAG-style retrieval. Optional Gemini API support can be added if an API key is configured.

12. Question: Do you need to say you used Gemini API?

Answer: No. I should not say Gemini API was used unless it is configured and used in the final system. The correct explanation is that the chatbot uses local RAG-style retrieval from existing project and product data. Gemini can be mentioned only as a future improvement or optional extension.

13. Question: Why did you not use Gemini API? Is RAG better than Gemini?

Answer: RAG and Gemini are not competitors. Gemini is an AI model, while RAG is a method. I did not use Gemini API because my chatbot answers are project-specific and can be handled from existing project/product data. Local RAG-style retrieval is lightweight, explainable, cost-free, and does not depend on external API access. Gemini can be added later to generate more natural answers from retrieved context.

14. Question: How did you implement RAG here?

Answer: I implemented RAG by creating a local knowledge base from project information and product data. When the user asks a question, the chatbot normalizes and tokenizes the text, searches project/product documents, scores the best matches, retrieves the most relevant context, and prepares an answer from that context. This is a lightweight local RAG implementation.

15. Question: What questions can the chatbot answer?

Answer: It can answer project questions, product questions, inventory questions, demand forecast questions, and setup questions. Example questions include: What is Smart Commerce? Which products are under 500 taka? Tell me about FastCharge Power Bank stock price and sales. How does low-stock alert work? What technologies are used for demand forecasting? How does the chatbot work?

16. Question: What is the system architecture?

Answer: Smart Commerce follows a client-server architecture. The frontend is built with Next.js, React, TypeScript, and Tailwind CSS. The backend is built with Node.js, Express.js, TypeScript, MongoDB, and Mongoose. MongoDB stores users, roles, products, categories, orders, inventory logs, and audit logs. The backend also runs a Python forecasting script for demand prediction.

17. Question: What is the architecture workflow?

Answer: User requests go to the Next.js frontend. The frontend sends API requests to the backend. The backend processes business logic and communicates with MongoDB. Responses return to the frontend. For demand forecasting, the backend prepares order/product sales history and sends it to the Python forecasting script, then shows results in the admin dashboard.

18. Question: Which SDLC model did you follow?

Answer: I followed the Iterative and Incremental SDLC model. I developed the project feature by feature instead of building everything at once. Each module was implemented, tested, improved, and then the next module was added.

19. Question: Why is Iterative and Incremental better than Waterfall for this project?

Answer: This project has many modules, such as products, authentication, cart, checkout, order tracking, inventory, low-stock alert, demand forecasting, chatbot, PDF receipt, and Docker setup. These were added step by step. Iterative and Incremental is suitable because it allows testing and improving each feature before adding the next one.

20. Question: What are the development iterations?

Answer: Iteration 1: basic storefront and products. Iteration 2: authentication and role-based dashboards. Iteration 3: cart, checkout, and order tracking. Iteration 4: inventory and low-stock alert. Iteration 5: demand forecasting. Iteration 6: chatbot and PDF receipt. Iteration 7: testing, Docker, and final improvements.

21. Question: How did you test the project?

Answer: I tested it using manual testing, API testing, build testing, and Docker testing. I tested login, role-based access, product browsing, cart, checkout, order tracking, PDF receipt, inventory update, low-stock alert, demand forecast, and chatbot responses. I also tested TypeScript, linting, production build, and Docker Compose.

22. Question: Which commands were used for testing?

Answer: I used commands such as npm run lint, npx tsc --noEmit, npm run build, and docker compose up --build -d. I also tested endpoints like /api/health, /api/chatbot, /api/products, /api/products/inventory, and /api/analytics/demand-forecast.

23. Question: Why is the project named Smart Commerce?

Answer: It is named Smart Commerce because it is not only a normal e-commerce website. It combines online shopping with smart business support features such as role-based dashboards, inventory intelligence, low-stock alerts, demand forecasting, analytics, chatbot support, order tracking, and PDF receipts.

24. Question: What makes it different from a normal e-commerce system?

Answer: A normal e-commerce system mainly focuses on product browsing, cart, and order placement. Smart Commerce also supports admin/vendor decision-making with inventory control, low-stock alerts, analytics, demand forecasting, audit activity, and chatbot support.

25. Question: How did you run the project with Docker?

Answer: I fixed the backend .env file because a normal sentence was accidentally added inside the env file. Docker env files must contain only KEY=value lines or comments. After fixing it, I ran docker compose up --build -d. The containers smart-commerce-frontend, smart-commerce-backend, and smart-commerce-mongo started successfully.

26. Question: What was the Docker error and why did it happen?

Answer: The error was: unexpected character '?' in variable name. It happened because the sentence 'when i click to login show this message why ?' was accidentally written inside backend/.env. Docker tried to read that sentence as an environment variable, so Compose failed.

27. Question: What are the important project URLs after running Docker?

Answer: Frontend: http://localhost:3000. Backend health check: http://localhost:5000/api/health. Chatbot API: http://localhost:3000/api/chatbot.